

iClosed Web Application — Project Review

Prepared for: Client Presentation

Date: 29 May 2026

Application: iClosed — Real-estate legal closing portal

Stack reviewed: Next.js 16 · React 19 · TypeScript · Supabase (PostgreSQL + Auth) · Vercel Blob · Resend

1. Executive Summary

iClosed is a well-structured, modern web application that manages the end-to-end real-estate closing experience — lead intake, task and milestone tracking, document upload, e-signature of retainer agreements, and co-purchaser collaboration. The codebase is clean and the core identity logic (how a logged-in user is matched to their client record) is genuinely robust.

However, this review found a **systemic authorization gap**. The application reliably knows *who* a user is, but in most places it does ****not enforce *what* that user is allowed to see or change.**** As a result, sensitive client data — ****including uploaded identification documents (passports, driver's licenses), personal information, deal pricing, and the entire admin dataset**** — can currently be **read and in some cases modified or deleted without logging in**.

Headline risk: *Sensitive client data, including uploaded ID documents, is currently retrievable and modifiable without authentication.*

The good news: the root cause is a single, well-understood design decision (described in §3), and the fixes follow a clear, low-to-moderate-effort path. None of the findings indicate a rewrite is needed — they are additions of access-control checks to existing, otherwise-sound code.

Risk Dashboard

Severity	Count	Examples
----------	-------	----------

Severity	Count	Examples
🔴 Critical	5	Unauthenticated admin API; ID documents readable/deletable by anyone; task tampering
🟡 High	3	Further IDOR endpoints; open CORS; database-level isolation disabled
🟢 Medium	7	No rate limiting; logic bugs; missing error handling; no input validation
🟡 Low	4	No automated tests/CI; hard-coded values; inconsistent error formats

Overall verdict

Not yet ready for production handling of real client PII in its current state. With the **Phase 0 + Phase 1** remediation (estimated ~2–3 weeks of focused work, see §6), the most serious exposures can be closed. The application is a strong foundation — the gaps are fixable, not fundamental.

2. Scope & Methodology

What was reviewed:

- The complete repository: all API route handlers, authentication and data-access layers, key React UI flows, the SQL migration files, and project configuration.
- Three focus areas, as requested: **security threats**, **missing/incorrect logic**, and **code quality & architecture**.

How findings were verified:

- Every finding marked **VERIFIED** in this document was confirmed by opening the actual source file and reading the exact code — file paths and line numbers are cited so your development team can go straight to them.
- The git history was checked to determine whether secret keys were ever committed.

****What was *not* covered (recommended as follow-up):****

- A live penetration test against the running application.
 - An audit of the actual Supabase **Row-Level Security (RLS)** policies configured in the live database (this review can only see the application code, not the database's own rules — see §3).
 - A dependency vulnerability (CVE) scan of the full package tree.
-

3. The Root Cause (Please read this first)

Almost every security finding in this report traces back to **one architectural decision**.

Every server-side API route in the application connects to the database using the

Supabase "service-role" client ([src/lib/supabaseAdmin.ts](#)). The

service role is an all-powerful key that **bypasses Supabase's Row-Level Security (RLS)** — the database feature that would normally guarantee "User A can only ever see User A's rows," *even if the application code has a bug.*

This has two consequences:

1. **No safety net.** Because RLS is bypassed, the database will hand back *any* row the code asks for. Security therefore depends **entirely** on each individual API route remembering to check "does this data belong to the person making the request?" — and many of them do not.
2. **Identity is solved; authorization is not.** The function that identifies the logged-in user, [src/lib/getAuthClient.ts](#), is genuinely well-built — it has a robust 4-step fallback to correctly match a user to their client record even in tricky co-purchaser and renamed-account scenarios. **The problem is that most routes don't *use* this identity to filter their queries.** Several routes accept an `email`, a `lead_id`, or a task `id` straight from the URL and return data with no login required at all.

In one sentence: *The app knows who you are, but mostly doesn't check whether you're allowed*

to have what you're asking for — and the database has been configured not to stop it either.

Fixing this is the spine of the remediation plan in §6: add an ownership check to each route, and turn the database safety net (RLS) back on as defense-in-depth.

4. Security Findings

Each finding lists **what it is**, **how it could be exploited**, **what data is at risk**, its **severity**, and the **recommended fix**.

🔍 CRITICAL

C1 — Admin API has no authentication

File: `src/app/api/admin/leads/route.ts` (and the wider `admin/*` folder) · **VERIFIED**

- **What:** The admin endpoints have no login or role check whatsoever. `GET /api/admin/leads` returns **every lead in the system** — full name, email, phone, address, and price. The same no-auth pattern applies to `admin/convert-lead`, `admin/clients/[id]/name`, `admin/clients/merge`, and `admin/link-co-purchaser`. Several also send `Access-Control-Allow-Origin: *`, inviting cross-site calls.
- **Exploit:** Anyone who knows (or guesses) the URL can download the entire customer database, or rename/merge client accounts.
- **Data at risk:** The complete customer dataset; account integrity (merge can effectively hand one person's deals to another).
- **Fix:** Require an authenticated admin session on every `admin/*` route (see `requireAdmin()`, §6 Phase 1). Restrict CORS to known origins.

C2 — Uploaded ID documents are readable and deletable by anyone (IDOR)

File: `src/app/api/lead-identification-docs/route.ts` · **VERIFIED**

- **What:** `GET` returns identification documents for *any* `lead_id` supplied in the URL, with no login or ownership check. `DELETE` removes *any* document by its `id`, also unchecked.
- **Exploit:** By iterating IDs, an attacker can harvest scanned passports / driver's licenses for all clients, or delete a client's submitted documents.
- **Data at risk:** Government-issued identity documents — among the most sensitive PII the firm holds, with direct identity-theft and regulatory implications.
- **Fix:** Require authentication; confirm the lead/document belongs to the requesting client (or an admin) before returning or deleting.

C3 — Deal details exposed via an email parameter

File: `src/app/api/deals/route.ts` (lines 55–104) · **VERIFIED**

- **What:** The primary, session-authenticated path of this endpoint is correctly written. But it then falls back to a `?email=` lookup that returns a client's **entire deal set** to anyone who supplies that email — no login required. (A `?lead_id=` fallback behaves similarly.)

- **Exploit:** Knowing a client's email (often easy to guess or find) yields their file numbers, property addresses, prices, and closing dates.
- **Data at risk:** Transaction details and PII for any client.
- **Fix:** Remove the unauthenticated `email/lead_id` fallbacks, or gate them behind an authenticated/admin context.

C4 — Any task can be completed/modified without authorization

File: `[src/app/api/tasks/[id]/route.ts](src/app/api/tasks/[id]/route.ts)` · **VERIFIED**

- **What:** `PATCH /api/tasks/[id]` updates a task — marking it complete, setting document URLs, and advancing the deal's milestones — with **no check** that the task belongs to the caller.
- **Exploit:** Anyone can mark other clients' tasks complete or attach/replace document URLs, corrupting the closing workflow and triggering downstream milestone emails.
- **Data at risk:** Workflow integrity for every deal; potential for false "completed" states on legal-process steps.
- **Fix:** Resolve the caller's client via `getAuthClient()` and verify the task's deal belongs to them before mutating.

C5 — Production service-role key stored in a plaintext backup file

File: `.env.local.prod-backup` (on the developer machine) · **VERIFIED (with correction)**

- **What:** A backup file holds the **production** Supabase service-role key (full database bypass), plus the Resend (email) and Vercel Blob (storage) tokens, in plaintext.
- **Correction to note for the client:** These `.env` files are **correctly excluded from git** (`.gitignore` contains `.env*`) and a history check confirmed they were **never committed**. So this is **not** a public source-code leak. The real risk is **key-handling hygiene** — a production secret that grants total database access living unencrypted in a casually-named backup file on a workstation (easily copied, emailed, or synced to cloud backup by accident).
- **Exploit:** Anyone who obtains the file gets unrestricted read/write to the production database.
- **Fix (Phase 0):** Delete the on-disk backup, and **rotate** the service-role, Resend, and Blob keys as a precaution. Keep production secrets only in the hosting provider's secret store.

🔴 **HIGH**

H1 — Additional IDOR endpoints (no ownership check)

VERIFIED (pattern confirmed on representative files)

- `GET /api/task-responses` returns submitted form answers by `task_id`;

`/api/uploadblobstorage` and `/api/blob/save-doc-metadata` write documents against a `lead_id`; `/api/intake/mark-aps-uploaded` advances workflow — all by an ID with no ownership check.

- **Fix:** Same as C2/C4 — authenticate and verify ownership.

H2 — Open CORS on sensitive endpoints

- Several `admin/` routes return `Access-Control-Allow-Origin:`, which combined with the missing auth (C1) makes them callable from any website.
- **Fix:** Restrict to the application's own origin(s).

H3 — Database-level isolation (RLS) is bypassed everywhere

- The service-role client (§3) means the database enforces no row-level isolation. Even after the code-level checks are added, there is no second line of defense.
- **Fix:** Enable and audit Supabase RLS so the database independently enforces "users see only their own rows" (Phase 2).

🔗 MEDIUM / 🔗 LOW (security)

ID	Finding	Severity
M1	No rate limiting on auth, data, or upload endpoints — enables brute force, enumeration, and bulk data exfiltration.	Medium
M2	No input/type validation on query params (IDs are not validated as UUIDs); malformed input can leak structural detail via error messages.	Medium
M3	Unbounded queries / no pagination — endpoints return all rows at once, a PII-exposure and performance concern as data grows.	Medium

5. Logic & Correctness Findings

These are bugs in behaviour (not security), all **VERIFIED** in source.

ID	Finding	File	Severity	
L1	Operator-precedence bug. <code>`includes("sign")` <code>&& <code>includes("virtually") <code>\</code> </code></code></code>	<code>\</code>	includes("person")` mis-binds, so it matches <i>any</i> label containing "person" (e.g. "personal information"). The signing-method field resolves incorrectly.	PersonalInform
L2	Duplicate identical query. <code>siblings</code> and <code>updatedSiblings</code> run the exact same database query back-to-back; <code>siblings</code> is never used. A wasted round-trip, and the "re-fetch to get updated state" comment is misleading (nothing changed between the two).	<code>task-responses/route.ts:186</code>	Medium	
L3	Fire-and-forget sync. Shared-task synchronisation (for co-purchaser / Purchase-&-Sale families) is launched without <code>await</code> and only <code>.catch(log)</code> . If it fails, linked family members silently see stale/incomplete data with no error	<code>[tasks/[id]/route.ts:60](src/app/api/tasks/[id]/route.ts#L60)</code>	Medium	

ID	Finding	File	Severity
	surfaced.		
L4	Unchecked bulk writes. The task insert in lead conversion and the side-tagging updates ignore their error results and proceed regardless, so a failed write passes silently.	<code>convertLead.ts</code> · <code>tasks/route.ts</code>	Medium
L5	Milestone-advancement race. Milestone completion uses a check-then-update pattern with no locking; two tasks completing at once can double-advance or skip a milestone.	<code>syncSharedTask.ts</code> + task routes	Medium
L6	Unvalidated normalization. Unknown citizenship values pass through unchanged, allowing typo/garbage values to be persisted.	<code>task-responses/route.ts</code>	Low

6. Code Quality & Maintainability

- **Duplicated milestone-advancement logic** appears in three places (`([tasks/[id]]/route.ts)(src/app/api/tasks/[id]/route.ts), task-responses/route.ts, syncSharedTask.ts`). This should be a single shared helper — keeping three copies in sync is error-prone (and is part of why L5 exists in multiple paths).
 - **Near-zero automated tests.** Only three component tests exist; there is **no coverage** for API routes, authentication, lead conversion, or task syncing — the highest-risk business logic — and **no CI pipeline**. Regressions are likely to reach production unnoticed.
 - **Lost design documentation.** The explanatory JSDoc in the in-progress `findFamilySharedTaskPeers.ts` was removed, which will make the cross-side matching logic harder for future maintainers to safely change.
 - **Minor:** inconsistent API error-response shapes (`{success,error}` vs `{error}`), hard-coded contact phone/email in UI, and broad `any` usage / unsafe type casts in a few places.
-

7. Remediation Roadmap

A phased plan — earlier phases address the highest risk for the least effort.

Phase 0 — Immediate (a few days)

- **Rotate** the Supabase service-role, Resend, and Blob keys; **delete** the `.env.local.prod-backup` file from local disk. (C5)
- Add an **admin authentication guard** to every `admin/` route. (C1)*
- **Lock down CORS** to the application's own origins. (H2)

Phase 1 — Authorization (1–2 weeks)

- Introduce shared guards — `requireClient()` and `requireAdmin()` — built on the existing, already-solid `getAuthClient.ts`.
- **Scope every data route** to the resolved client, and **add ownership checks** to all IDOR endpoints (ID documents, deals, tasks, task-responses, uploads). (C2, C3, C4, H1)
- Remove or gate the unauthenticated `email` / `lead_id` fallbacks. (C3)

Phase 2 — Defense in depth (1–2 weeks)

- **Enable and audit Supabase RLS** so the database enforces isolation even if a route is missed. *(H3)*
- Add **rate limiting** and **UUID/input validation**. *(M1, M2)*
- Add **pagination** to list endpoints. *(M3)*

Phase 3 — Correctness & quality (ongoing)

- Fix logic bugs **L1–L6**.
- Extract the **shared milestone-advancement helper**.
- Add **automated tests** (API routes, auth, lead conversion, task sync) and a **CI pipeline**.

Phase	Focus	Effort	Priority
0	Secret rotation + admin lockdown	~2–3 days	🔴 Do now
1	Per-route authorization	~1–2 weeks	🔴 Critical
2	RLS, rate limiting, validation	~1–2 weeks	🔴 High
3	Bug fixes, refactor, tests/CI	Ongoing	🟡 Medium

8. Technical Appendix

8.1 Findings reference table

ID	Severity	Location	Issue	Recommendation
C1	Critical	<code>api/admin/*/route.ts</code>	Admin API has no auth; open CORS	Require admin session; lock CORS

ID	Severity	Location	Issue	Recommendation
C2	Critical	api/lead-identification-docs/route.ts	ID docs readable/deletable by anyone (IDOR)	Auth + ownership check
C3	Critical	api/deals/route.ts:55	Deals exposed via ?email= fallback	Remove/gate fallback
C4	Critical	api/tasks/[id]/route.ts	Task PATCH has no ownership check	Verify task→deal→client ownership
C5	Critical	.env.local.prod-backup	Prod service-role key in plaintext backup (gitignored, not committed)	Delete file; rotate keys
H1	High	task-responses, uploadblobstorage, blob/save-doc-metadata, intake/mark-aps-uploaded	Further IDOR	Auth + ownership check
H2	High	api/admin/*	Access-Control-Allow-Origin: *	Restrict origins
H3	High	lib/supabaseAdmin.ts (used everywhere)	Service role bypasses RLS — no DB safety net	Enable/audit RLS
M1	Medium	All endpoints	No rate limiting	Add rate limiting
M2	Medium	Query params	No input/UUID validation	Validate inputs

ID	Severity	Location	Issue	Recommendation
M3	Medium	List endpoints	No pagination / unbounded queries	Paginate
L1	Medium	<code>PersonalInformationDrawer.tsx:109</code>	Operator-precedence bug	Add parentheses
L2	Medium	<code>task-responses/route.ts:186</code>	Duplicate identical query	Remove dead query
L3	Medium	<code>tasks/[id]/route.ts:60</code>	Fire-and-forget sync, errors swallowed	Await / surface errors
L4	Medium	<code>convertLead.ts</code> , <code>tasks/route.ts</code>	Unchecked bulk writes	Check write results
L5	Medium	<code>syncSharedTask.ts</code> + task routes	Milestone-advancement race	Centralize + guard
L6	Low	<code>task-responses/route.ts</code>	Unvalidated normalization persists garbage	Validate enum
Q1	Low	3 files	Duplicated milestone logic	Extract shared helper
Q2	Low	Repo-wide	Near-zero tests, no CI	Add tests + CI
Q3	Low	Repo-wide	Inconsistent error shapes, hard-coded values	Standardize

8.2 Architecture summary (for the development team)

- **Stack:** Next.js 16 (App Router) · React 19 · TypeScript · Supabase (PostgreSQL + Auth, JWT in HTTP-only cookies via `@supabase/ssr`) · Vercel Blob (document storage) · Resend (email) · TailwindCSS · pdf-lib (retainer PDFs). Deployed on Vercel.
 - **Domain:** Real-estate legal closings. Customers submit a lead (purchase / sale / refinance), which an admin converts into a *deal*; the deal carries *milestones* (closing phases) and *tasks* (action items), driven by reusable *templates*. Supports dual-sided **Purchase & Sale** deals and **co-purchaser families** with shared, synchronised tasks. Includes retainer e-signature and AI-assisted document identification.
 - **Core entities:** `clients`, `leads`, `deals`, `milestones`, `tasks`, `task_templates`, `stage_templates`, `task_responses`, `retainer_signatures`, document tables (Vercel Blob).
 - **Auth/data layers:** `getAuthClient.ts` (identity — well built), `supabaseAdmin.ts` (service-role client — the root-cause concern of §3), `supabaseClient.ts` (anon client).
-

This document reflects a static review of the application source code as of the date above. A live penetration test and an audit of the production database's RLS policies are recommended as follow-up to confirm the runtime posture.